The Journal of Open Source Software

PVade: A Python package for simulating wind loading and aerodynamic stability in solar-tracking PV arrays

Walid Arsalane[1¶], Ethan Young[1], Brooke Stanislawski[1], Xin He[1], Matthew Emes[1], Martin Springer[1], and Andrew Glaws[1]

1 National Laboratory of the Rockies, Golden, CO, United States ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 05 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Wind-driven degradation of single-axis tracking photovoltaic (PV) systems encompasses a range of different phenomena, from small-scale fluctuating pressures that can exacerbate cracks to large-scale "torsional galloping" events which can lead to broken glass and catastrophic damage to tracker hardware. Modern PV panels and mounting structures are particularly susceptible to these phenomena, in large part due to industry trends toward larger module areas, thinner layers of encapsulating glass, and lightweight mounting components. Both the small- and large-scale dynamic responses to wind loading are fundamentally driven by the same multi-physics interaction; namely, how atmospheric flows induce loads on the panel surface and cascade through the array, generating non-uniform deformation and displacement in the structure as a function of both operational (e.g., tracker tilt angle) and design (e.g., panel frame stiffness) choices.

To address these issues and provide actionable guidance for system owners, we developed PV Aerodynamic Design Engineering (PVade) (Young et al., 2023), an open-source, HPC simulation framework capable of resolving the full fluid-structure interaction (FSI) between multiple rows of compliant PV hardware and realistic wind loading. PVade predicts pressure distributions on the panel surfaces, inertial dynamics of hardware elements, and predictions of aeroelastic instabilities across a range of different weather conditions, operating choices, and design choices. These outcomes provide guidance on which tracker tilt angles are more or less stable in a given weather condition and feedback on the efficacy of strengthening measures. Additionally, PVade outputs (e.g., time-varying pressure maps) can be used as part of a larger, co-simulation framework to predict the drivers of microcrack growth from a micro-scale model subjected to accurate wind loading boundary conditions.

Statement of need

Wind-induced mechanical degradation remains one of the most difficult reliability challenges for PV systems. Existing computational approaches either treat panels and mounting structures as rigid bodies, approximate multi-row effects with periodic boundary conditions, or require a prohibitively high computational cost to capture the coupled FSI between wind and hardware. This limits their ability to capture fluttering and torsional galloping events, ignores differences between perimeter and interior effects, or fails to capture the cascade of waking and complex loading conditions associated with interior rows with multiple upstream neighbors, respectively. Researchers and system owners alike need realistic stressor inputs (e.g., pressure, inertial dynamics, and deformations) to integrate into mechanical module models for crack weathering, glass breakage, and wind stow and deployment guidance. Industry partners need tools to study layout choices, tracker stiffness, panel thickness, and inter-row spacing to mitigate wind-related degradation. Importantly, prior work within the Durable Module Materials (DuraMAT)

DRAFT

Consortium (Barnes et al., 2023) has demonstrated that mounting hardware dynamics play a critical role in amplifying small motions into destructive rotations, but there is no existing open-source, HPC-capable modeling framework that can capture these FSI instabilities across multi-row PV arrays. This software fills that gap by providing an accurate, scalable framework that integrates high-fidelity CFD, structural dynamics, and mesh motion into a single accessible tool.

State of the field

Outside DuraMAT and the field of PV, open-source computational fluid dynamics (CFD) solvers exist, but adapting them to the fluid and structure boundary conditions defined by single-axis tracking systems and adapting the mesh for the specifics of large-scale fluttering is a significant barrier to entry for new users.

Software design

`PVade` is designed as a modular, component-based multiphysics simulation framework built on top of `FEniCSx` for finite element discretization and solution of partial differential equations (Alnaes et al., 2014; Baratta et al., 2023; Scroggs et al., 2022) and `Gmsh` for geometry construction and mesh generation (Geuzaine & Remacle, 2009) . The architecture is organized into five primary modules—Geometry, I/O, Fluid, Structure, and FSI—which collectively define the core structure and execution workflow of the framework. This modular design enables separation of concerns between geometry generation, physics definition, coupling, and data management, improving maintainability and extensibility.

The Geometry module is responsible for converting user-defined inputs into computational meshes. `PVade` supports multiple geometries including two- and three-dimensional cylinders (used for benchmarking and verification), photovoltaic (PV) modules in 2D and 3D, a 2D flag configuration, and a 3D heliostat model. The geometry subsystem is centered around a MeshManager component, which either reads externally generated meshes in XDMF format or constructs meshes from scratch. Mesh creation begins with a domain-specific geometry definition file corresponding to each supported configuration. Once the geometric domain is defined, mesh generation proceeds through facet and volume initialization, element length scale definition, and boundary tagging. The module outputs both XDMF mesh files and distributed mesh objects compatible with `FEniCSx` for subsequent physics solves.

The I/O module manages input parsing, data formatting, logging, and solution output. It controls the transfer of data between physics modules, supports interpolation required for mesh motion in multiphysics simulations, and handles generation of mesh and solution files in XDMF format. All simulation inputs are provided through YAML configuration files, while output files—including mesh and solution fields—are written in XDMF format to ensure compatibility with standard post-processing tools.

The Fluid module defines all fluid-related variables and governing equations. This includes formulation of the variational statements, boundary condition specification, matrix assembly, and solution of the discretized fluid PDE system using the finite element capabilities of `FEniCSx`.

The Structure module encapsulates structural physics models, including elasticity, thermal analysis, and modal analysis. Boundary conditions are defined and applied according to the selected structural physics model. A central structural driver coordinates the interaction between different structural analyses while maintaining modular separation of each physical formulation.

The FSI module manages mesh motion and coupling in fluid–structure interaction simulations. It coordinates data transfer between fluid and structural domains, leveraging the I/O module

DRAFT

for consistent interpolation of field variables between nonconforming meshes when necessary. This module ensures synchronization of solution fields and mesh updates during coupled solves.

The execution of PVade is coordinated through a main driver script, `pvade_main.py`, which orchestrates interactions between all modules. Simulations are launched from the command line and require a YAML configuration file specifying geometry, physics, solver, and runtime parameters. Default input templates are provided for supported geometries and physics configurations, and parameters may be overridden via command-line arguments to facilitate testing and parametric studies.

The runtime workflow consists of input validation, mesh construction or loading, physics initialization (including variational form definition and boundary condition setup), solver configuration and matrix assembly, and finally the solve phase. During the solve stage, each active physics module is executed, coupled data are propagated as required, and solution fields are written to disk. All physics models are discretized using finite element methods within FEniCSx, which internally leverages scalable linear algebra and parallel communication backends for high-performance execution.

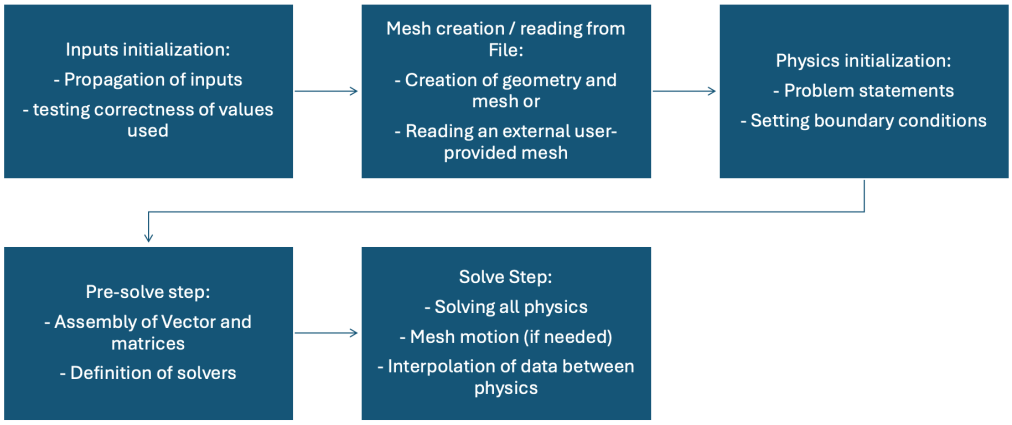


Figure 1: Flowchart view of PVade structure.

PVade follows a modular design philosophy that promotes extensibility and maintainability. The framework is structured such that geometry definition, physics modules, and input/output handling are encapsulated within independent components. This separation enables the addition of new physics models, geometries, or I/O capabilities without major restructuring of the existing codebase. The modular organization minimizes cross-dependencies between subsystems and supports incremental feature development.

New physics models can be incorporated within the existing Fluid or Structure modules, depending on the governing equations and domain of application. Each physics model defines its own variational formulation, material properties, and boundary condition logic. Integration of a new physics capability is achieved by registering it within the corresponding FluidMain or StructureMain driver, which coordinates initialization, assembly, and solution procedures. This approach preserves a consistent execution workflow while allowing expansion of the supported multiphysics capabilities.

PVade is designed to operate seamlessly across computing environments ranging from personal laptops to distributed-memory high-performance computing (HPC) clusters. The framework

relies exclusively on MPI-based parallelism and does not currently support OpenMP threading or GPU acceleration. To simplify deployment, an environment.yaml file is provided for installation via Conda or Mamba (e.g., Miniforge), ensuring reproducible dependency management. This environment specification enables straightforward setup across heterogeneous systems with minimal configuration effort.

A comprehensive testing strategy is implemented using the pytest framework. Unit and regression tests cover core functionality, including mesh generation, boundary condition application, variational form assembly, and physics solves. Automated testing ensures correctness across geometry configurations and physics models, reducing the likelihood of regression errors as new features are introduced.

Version control is managed through Git, with continuous integration (CI) workflows hosted on GitHub. All tests are automatically executed on both development and main branches upon code updates, ensuring stability and consistency before changes are merged into the primary codebase.

For reproducibility, PVade is currently installable via Miniforge using the provided Conda environment specification. Ongoing development efforts aim to further enhance portability through containerized distributions and potentially through the publication of a dedicated Conda package channel. These efforts are intended to streamline deployment and ensure long-term reproducibility across computational platforms.

Research impact statement

PVade has been used to quantify the wind load on agrivoltaics systems (Young & Krause, 2024), the motion of floating PV systems (Rasmussen et al., 2026), and the wind flow profile through single-axis tracking arrays (Stanislawski et al., 2026), all enabled through partnerships with industry. Additionally, PVade is being explored by renewable energy system insurers to quantify the risk associated for different weather events, system layouts, and hardware choices. The spatially varying load profiles from PVade are being used to inform new tracker testing standards, specifically, the specification of non-uniform mechanical loads. Finally, the outputs from PVade have been used by colleagues at the Colorado School of Mines to construct datasets which can be used to train fast, machine-learned surrogate models that can accurately predict structural deformations without the large computational expense.

AI usage disclosure

- Tool usage: the authors utilized GitHub Copilot, ChatGPT (GPT-5.6 Sol, GPT-5.6 Terra), Claude (sonnet 5).
- The nature and scope of assistance: Assist with documentation and manuscript refinement, through copy editing, grammar checks, and ensuring structural cohesiveness.
- Confirmation of review: We assert that we have reviewed and edited all AI-assisted outputs and made the core design decisions.

Acknowledgements

This work was authored by the National Laboratory of the Rockies for the U.S. Department of Energy (DOE), operated under Contract No. DE-AC36-08GO28308. Funding provided as part of the Durable Module Materials Consortium 2 (DuraMAT 2), funded by the U.S. Department of Energy Office of Critical Minerals and Energy Innovation Integrated Energy Systems Office, agreement number 38259. The views expressed in the article do not necessarily represent the views of the DOE or the U.S. Government. The U.S. Government retains and the publisher,

DRAFT

by accepting the article for publication, acknowledges that the U.S. Government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this work, or allow others to do so, for U.S. Government purposes.

References

Alnaes, M. S., Logg, A., Ølgaard, K. B., Rognes, M. E., & Wells, G. N. (2014). Unified form language: A domain-specific language for weak formulations of partial differential equations. *ACM Transactions on Mathematical Software*, *40*. https://doi.org/10.1145/2566630

Baratta, I. A., Dean, J. P., Dokken, J. S., Habera, M., Hale, J. S., Richardson, C. N., Rognes, M. E., Scroggs, M. W., Sime, N., & Wells, G. N. (2023). *DOLFINx: The next generation FEniCS problem solving environment*. preprint. https://doi.org/10.5281/zenodo.10447666

Barnes, T., Schelhas, L., Hansen, C., Steinman, L., & Jain, A. (2023). *Durable module materials (DuraMAT) consortium (final technical report)*. National Renewable Energy Laboratory (NREL), Golden, CO (United States). https://doi.org/10.2172/1962808

Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-d finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, *79*(11), 1309–1331. https://doi.org/https://doi.org/10.1002/nme.2579

Rasmussen, M., Emes, M., Young, E., & Nicholson, P. (2026). *Dynamic wind loading on compliant, floating PV arrays: Cooperative research and development (final report)*. National Laboratory of the Rockies (NLR), Golden, CO (United States); AccuSolar, LLC, Pompano Beach, FL (United States). https://doi.org/10.2172/3016091

Scroggs, M. W., Dokken, J. S., Richardson, C. N., & Wells, G. N. (2022). Construction of arbitrary order finite element degree-of-freedom maps on polygonal and polyhedral cell meshes. *ACM Transactions on Mathematical Software*, *48*(2), 18:1–18:23. https://doi.org/10.1145/3524456

Stanislawski, B., Young, E., & de Vries, N. (2026). *Modeling wind in agrivoltaics and its impact on eddy covariance flux measurements* [Poster]. National Laboratory of the Rockies (NLR). https://doi.org/10.66816/po2677779

Young, E., Arsalane, W., Stanislawski, B., He, X., Ivanov, C., Dana, S., & Deceglie, M. (2023). *PVade (PV Aerodynamic Design Engineering) [SWR-23-49]*. https://doi.org/10.11578/dc.20231208.1

Young, E., & Krause, D. (2024). *Simulate wind loading on RUTE SunTracker: Cooperative research and development (final report)*. National Renewable Energy Laboratory (NREL), Golden, CO (United States). https://doi.org/10.2172/2481679

DRAFT